

Functional Foundations of Graph Representation Learning: Construction, Transformations, and Feature Engineering

The paradigm of Geometric Deep Learning (GDL) and graph-based machine learning relies heavily on the efficient, accurate, and scalable construction and manipulation of graph-structured data. In the context of large-scale computational pipelines—ranging from molecular property prediction to road network extraction—the implementation of graph utility operations dictates the upper bounds of both analytical performance and expressive capability. This exhaustive research report details optimal, pure-function implementations for graph construction, feature computation, and Graph Neural Network (GNN) preprocessing. The analysis is specifically targeted at informing the development of utility atoms for `sciona-atoms` (general graph computation) and `sciona-atoms-dl` (GNN-specific operations).

The subsequent analysis addresses critical computational stages across an estimated seventeen Computational Directed Graph (CDG) environments. These include the extraction of molecular graphs from 3D coordinates (such as those seen in CHAMPS and Novozymes datasets), spatial nearest-neighbor construction from point clouds (IceCube Neutrino analysis), knowledge graph transformations (OGB WikiKG90M), spectral feature engineering (COVID-19 mRNA constraints), and the topological translation of skeletonized image data into robust network structures (SpaceNet3). By evaluating algorithmic complexities, architectural dependencies, and memory paradigms, this research delineates a rigorous blueprint for best-in-class utility atoms.

Architectural Paradigms and Data Contracts

A rigorous functional approach to graph processing requires strict data contracts, state isolation, and format standardization. The tension between different graph representations forms the foundational architectural consideration for any graph utility library. The operations required in GNN architectures and general graph theory traverse distinct memory and processing paradigms, demanding an optimized approach to data structures.

The Duality of Graph Representation

The optimal graph format depends intrinsically on the subsequent computational stage. Graph operations generally fall into two broad categories: spatial message-passing operations and spectral or algebraic operations. The representation must be tailored to the mathematical requirements of these categories.

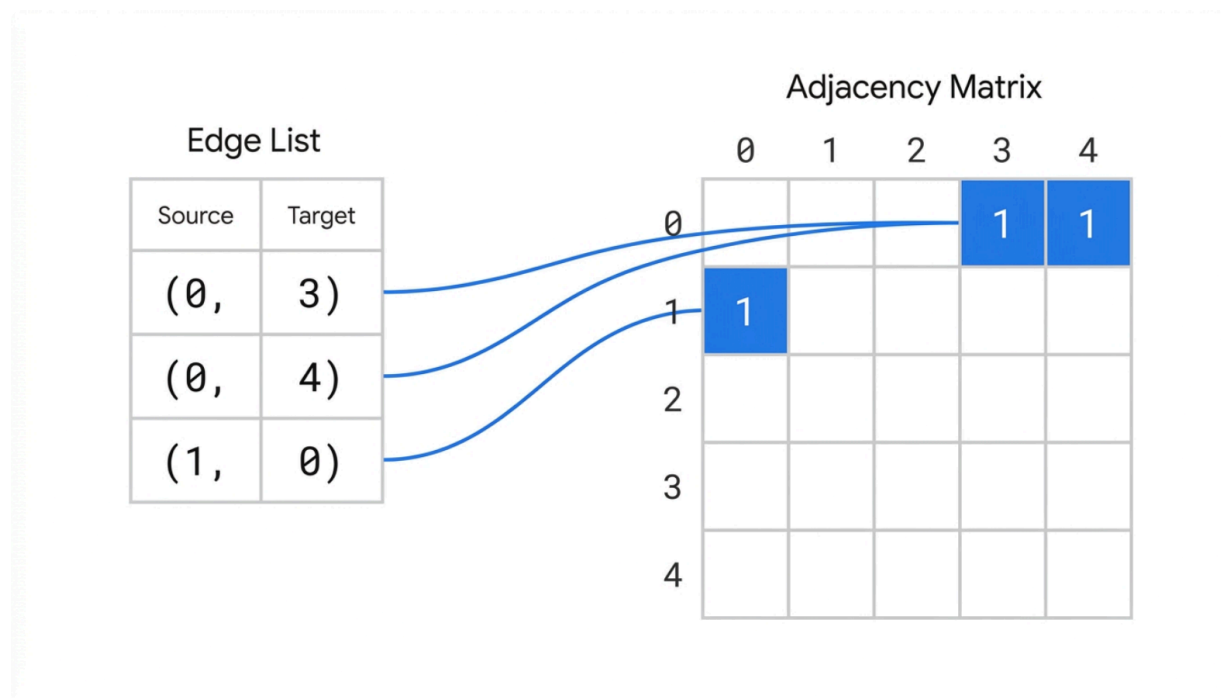
The Coordinate Format, colloquially known as the Edge List, represents the graph topology as a

contiguous array of shape $(2, E)$, where E is the number of edges. This format is the gold standard for spatial message-passing frameworks, including PyTorch Geometric (PyG). The Edge List is exceptionally memory-efficient for highly sparse graphs, such as molecular structures or road networks, scaling at $O(E)$ rather than $O(N^2)$. Furthermore, it allows for direct integer indexing into node feature matrices during hardware-accelerated scatter-gather operations. Message passing operations treat the graph as a routing mechanism, natively aligning with the Edge List paradigm.

Conversely, the Adjacency Matrix is represented as an (N, N) array, either dense or compressed into sparse formats like Compressed Sparse Row (CSR) or Compressed Sparse Column (CSC). This format is mathematically superior for spectral methods, eigenvalue decompositions, and global matrix operations.² Computations involving the Graph Laplacian, random walk transition matrices, and PageRank rely on treating the graph as a linear operator. In these scenarios, the topological structure must be subjected to matrix multiplication, an operation for which the Edge List is poorly suited without intermediate translation.

Both formats are strictly necessary within a comprehensive graph utility ecosystem. Atoms must be designed to natively support their most natural input format, while dedicated utility functions provide lossless, high-speed conversions between them.

Graph Duality: Edge List vs. Adjacency Matrix Formats



The structural translation of a sparse (2, E) edge list into a dense (N, N) adjacency matrix. Edge lists optimize memory for message passing, while matrices enable rapid linear algebraic operations.

Functional Contracts and Boundaries

To ensure determinism and portability across various accelerators (e.g., CPU, GPU, TPU), implementations within sciona-atoms must strictly adhere to the pure function paradigm. Pure functions enforce a stateless execution model, which is critical for distributed training and reproducible data generation.

The pure function boundary entails several architectural mandates. First, operations must not depend on or alter variables outside their local scope. Global state must be entirely eliminated. Second, hardware acceleration orchestration is left to higher-level frameworks. While PyTorch or JAX implementations might handle device placement natively, pure NumPy or SciPy atoms must operate strictly on host memory arrays, ensuring mathematical transparency.³ Finally, all hyperparameters, such as spatial radius thresholds, neighbor counts, or damping factors, must be passed explicitly as arguments rather than inferred from configuration files.

Contractual assertions must also be standardized across all implementations. For example, spatial parameters like k or radius must be strictly positive. The edge_index array must contain strictly valid integer indices bounded by the number of nodes $[0, N - 1]$, and adjacency

matrices must consistently conform to an (N, N) structural shape.

Spatial Graph Construction Algorithms

Spatial graph construction is the algorithmic process of defining topological connectivity based on the geometric coordinates of entities in a continuous space. This methodology is central to numerous CDG stages, including molecular modeling, point cloud processing (such as the IceCube Neutrino analysis), and the generation of interaction graphs from geographic embeddings.

K-Nearest-Neighbor (KNN) Topology Construction

The construction of a k -nearest-neighbor graph from a coordinate array is a heavily utilized preprocessing step across domains ranging from recommender systems to physical simulations. Given a set of points $X \in \mathbb{R}^{N \times F}$, the mathematical objective is to identify the k closest neighboring points for each point x_i under a specified distance metric, which is typically the Euclidean norm or a Minkowski parameter of $p = 2$.

The computational complexity of nearest-neighbor searches natively scales at $O(N^2)$ if executed via naive brute-force pairwise distance calculations. For massive graphs, this quadratic scaling is prohibitive. In standard NumPy and SciPy ecosystems, spatial partitioning data structures provide the optimal balance of speed and scalability. The k -d tree (k-dimensional tree) works by recursively partitioning F -dimensional data using hyperplanes, allowing the search space to be pruned geometrically.⁴

The `scipy.spatial.cKDTree` class is a highly optimized C++ implementation exposed to Python, offering $O(N \log N)$ construction time and logarithmic query time per node.⁴ Unlike alternative implementations—such as `sklearn.neighbors.NearestNeighbors`, which carries the significant overhead of the Scikit-Learn estimator API and extensive input validation checks⁶—the `cKDTree` provides direct, low-level access. This lack of overhead makes it highly suitable for integration as a pure functional wrapper.⁵

However, the efficacy of the k -d tree diminishes as the dimensionality of the features increases, succumbing to the curse of dimensionality where distance metrics lose their discriminatory power and the algorithm approaches $O(N)$ query time per point. For extremely high-dimensional embedding spaces (e.g., $F > 20$), or for very small values of N (e.g., $N < 1000$), brute-force calculation using highly optimized BLAS routines via

scipy.spatial.distance.cdist can outperform tree traversal due to memory locality and vectorization efficiency.⁸ Despite these edge cases, for general-purpose spatial coordinates (2D to 10D), the k -d tree remains the industry standard.

Name: knn_graph

Description: Build a k-nearest-neighbor graph from point or embedding features.

Source: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.html>

License: BSD-3-Clause

Concept type: graph_traversal

Signature: (features: NDArray, k: int, metric: str = "euclidean") -> tuple

Pure function boundary: node features and explicit graph parameters in, edge index and distances out; no training loop, global state, GPU state, or file I/O.

Contracts:

- require: features is a 2D array of shape (N, F)
- require: $0 < k < N$
- ensure: returned edge_index is of shape (2, N*k)
- ensure: returned edge_distances is of shape (N*k,)

Witness: Four 2D points forming a square with k=1; verify edges connect adjacent corners.

Dependencies: scipy

CDG stages covered: spatial_graph/knn, recommender/knn_graph, point_cloud_processing

Radius Graph Connectivity

A radius graph connects all nodes that fall within a specific geometric distance cutoff r . This algorithmic approach is fundamentally different from a KNN graph, as the degree of the resulting nodes is not fixed or guaranteed. Dense spatial regions will yield extremely high-degree nodes, whereas sparse spatial regions may yield completely disconnected components. This variance is crucial for physical systems where interactions decay exponentially with distance.

The scipy.spatial.cKDTree.query_ball_point method is recognized as the most robust analytical engine for this operation.⁵ By querying the initialized k -d tree with the coordinate array itself, the algorithm efficiently identifies all pairs bounded within the radius r . The atom's implementation must handle the translation of the query output, which returns an array of lists of varying lengths, into a flattened, contiguous (2, E) edge index suitable for message passing.

Name: radius_graph

Description: Connect all points within a specified spatial radius r.

Source:

https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.query_ball_point.html

License: BSD-3-Clause

Concept type: graph_traversal

Signature: (coordinates: NDArray, radius: float, p: float = 2.0) -> tuple

Pure function boundary: node coordinates and scalar radius in, edge index and distances out; completely stateless.

Contracts:

- require: coordinates is a 2D array of shape (N, F)
 - require: radius > 0.0
 - ensure: edge_distances <= radius for all returned edges
- Witness: Three collinear points separated by distance 1.0; query with radius 1.5 should yield edges between adjacent points but not the two ends.
- Dependencies: scipy
- CDG stages covered: spatial_graph/radius, point_cloud_processing

Molecular Graph Engineering from 3D Coordinates

Constructing molecular graphs from 3D atomic coordinates represents a domain-specific challenge prominently featured in datasets like CHAMPS, QM9, and Novozymes. The foundational architectural question is whether the graph should rely strictly on continuous spatial distances or whether it should incorporate discrete, explicit chemical bond types (e.g., single, double, or aromatic bonds) generated via cheminformatics libraries such as RDKit.

Analysis of state-of-the-art coordinate-based Graph Neural Networks—including architectures like SchNet, DimeNet, and GCPNet—indicates a decisive structural shift toward purely geometric representations.¹¹ These networks rely heavily on radial basis functions and directional or angular message passing applied to continuous spatial distances. By doing so, they bypass the necessity for explicit, rule-based bond typings, allowing the neural network to infer quantum interactions directly from proximity and angular geometry.¹¹

Integrating RDKit into a pure functional pipeline introduces significant architectural friction. It requires heavy dependencies, complex edge-case handling for varying chemical file formats, and fundamentally limits the atom strictly to cheminformatics applications rather than general physics simulations.¹⁴ Conversely, a pure distance-based cutoff is computationally superior, adheres strictly to the pure-function paradigm, and is universally generalizable across any

particulate system.³

By defining a physical cutoff threshold—typically between 1.5 to 2.0 Angstroms for covalent molecular systems—the adjacency matrix can be accurately reconstructed directly from Cartesian coordinates. Given that molecular graphs typically exhibit small node counts (often $N < 100$ atoms), vectorized distance matrix calculations via `scipy.spatial.distance.cdist` or native NumPy broadcasting provide maximum computational throughput.⁸ The `cdist` function evaluates the cross-distance between all pairs efficiently, and applying a boolean mask `< cutoff` instantly yields the valid edges.¹⁶

Name: `molecular_distance_graph`

Description: Construct a molecular graph from 3D atomic coordinates using a distance cutoff.

Source: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.distance.cdist.html>

License: BSD-3-Clause

Concept type: `graph_traversal`

Signature: (`coordinates: NDArray`, `elements: NDArray`, `cutoff: float`) -> tuple

Pure function boundary: Atomic coordinates, element arrays, and cutoff threshold in; edge indices, distances, and derived edge features out.

Contracts:

- require: `coordinates` is a 2D array of shape $(N, 3)$
- require: `elements` is a 1D array of shape $(N,)$
- require: `cutoff` > 0.0
- ensure: returned edge indices are valid integer indices < N
Witness: H₂O molecule coordinates; verify edges between O-H but not H-H under a 1.2Å cutoff.
Dependencies: `numpy`, `scipy`
CDG stages covered: `molecular_graph_construction`, `computational_chemistry`

Core Matrix-List Structural Conversions

The capacity to translate seamlessly and rapidly between dense adjacency matrices and coordinate edge lists is a vital prerequisite in the CDG lifecycle. Pure NumPy implementations are strictly preferable for this stage to avoid unnecessary dependency overhead from heavy graphing libraries like NetworkX or data manipulation libraries like Pandas. The conversions must operate strictly on array buffers to maintain execution speed.

Dense Adjacency Matrix to Sparse Edge List

Given an (N, N) adjacency matrix A , extracting the $(2, E)$ edge list is inherently a search

operation for non-zero elements. In pure NumPy, the `np.argwhere` function serves as the most performant method for dense matrices. Executing a conditional search such as

`np.argwhere(adj_matrix == 1)` or `np.argwhere(adj_matrix != 0)` returns an $(E, 2)$ array of indices identifying the source and target nodes. This array is subsequently transposed to achieve the PyG-standard $(2, E)$ format.¹⁸ For heavily weighted continuous graphs where any non-zero value represents an edge, `np.nonzero` yields identical results with parallel row and column index arrays that can be rapidly stacked.¹⁸

Name: `adjacency_to_edge_list`

Description: Convert an $N \times N$ adjacency matrix to a $2 \times E$ edge list format.

Source: <https://numpy.org/doc/stable/reference/generated/numpy.argwhere.html>

License: BSD-3-Clause

Concept type: `graph_traversal`

Signature: `(adj_matrix: NDArray) -> NDArray`

Pure function boundary: 2D adjacency array in, 2D edge list array out.

Contracts:

- require: `adj_matrix` is a 2D array of shape (N, N)
 - ensure: output is of shape $(2, E)$
- Witness: A 3×3 identity matrix; verify output is `[, ]`.

Dependencies: `numpy`

CDG stages covered: `graph_format_conversion`, `spectral_to_spatial`

Edge List to Adjacency Matrix Reconstruction

The inverse topological operation—mapping a $(2, E)$ edge list back to an (N, N) adjacency matrix—requires initializing a zero-filled matrix and accumulating edge weights at specific coordinates. Standard array indexing techniques (e.g., `A[sources, targets] += 1`) fail to correctly accumulate values when the edge list contains parallel edges (multigraphs), as standard indexing buffers the operations rather than processing them sequentially.

To resolve this, the NumPy `np.add.at` method provides an unbuffered, highly efficient mechanism to accumulate edge weights directly into the target matrix, correctly handling parallel edges by executing the addition in-place for every specified index combination.¹⁹

Name: `edge_list_to_adjacency`

Description: Convert a $2 \times E$ edge list to a dense $N \times N$ adjacency matrix.

Source: <https://numpy.org/doc/stable/reference/generated/numpy.add.html>

License: BSD-3-Clause

Concept type: graph_traversal

Signature: (edge_list: NDArray, num_nodes: int) -> NDArray

Pure function boundary: 2D edge list and integer node count in, 2D adjacency array out.

Contracts:

- require: edge_list is a 2D array of shape (2, E)
- require: max(edge_list) < num_nodes
- ensure: output is of shape (num_nodes, num_nodes)
Witness: Edge list [,] with num_nodes 2; verify symmetric 2x2 matrix with off-diagonals equal to 1.
Dependencies: numpy
CDG stages covered: graph_format_conversion, spatial_to_spectral

Spectral Topology Engineering

Extracting latent structural signals from graph topology is critical for generating rich node-level and graph-level embeddings before neural processing begins. These spectral operations treat the graph not merely as a set of connections, but as a discrete algebraic structure capable of undergoing linear transformations.

The Symmetric Normalized Graph Laplacian

The Graph Laplacian is the absolute cornerstone of spectral graph theory. It is utilized extensively in manifold learning, spectral clustering algorithms, and architectures modeling constraints, such as the COVID-19 mRNA vaccine degradation structures.²

The combinatorial, unnormalized Laplacian is mathematically defined as $L = D - A$, where D is the diagonal degree matrix and A is the adjacency matrix.²⁰ However, deep learning models overwhelmingly favor the symmetric normalized Laplacian, defined mathematically as:

$$\mathcal{L} = I - D^{-1/2} A D^{-1/2}$$

Normalization maps the spectrum of eigenvalues to the strict bounds of $[-1, 1]$. This mathematical constraint is imperative; without it, repeated application of the graph shift operator during polynomial filter approximations in Spectral Graph Convolutional Networks (GCNs) would lead to exploding gradients and numerical instability.²

The pure NumPy implementation of the symmetric normalized Laplacian requires precise handling of isolated nodes (where the node degree is 0) to avoid division by zero errors. The algorithmic process involves creating the degree matrix, extracting its main diagonal, safely applying the inverse square root only to non-zero elements, and performing the sequence of

matrix multiplications.²⁰

To demonstrate the mathematical effect of this transformation, consider a similarity matrix representing three highly connected nodes with varying edge weights. The transformation normalizes the diagonal strictly to 1.0, while scaling the off-diagonal connections inversely by the square root of the connecting nodes' degrees.

Matrix State	Node 0	Node 1	Node 2
Input Adjacency (A)			
Node 0	1.000	0.500	0.200
Node 1	0.500	1.000	0.500
Node 2	0.200	0.500	1.000
Normalized Laplacian ($\mathcal{L}$)			
Node 0	1.000	-0.597	-0.285
Node 1	-0.597	1.000	-0.597
Node 2	-0.285	-0.597	1.000

Data indicates that the normalization process forces the main diagonal elements to 1.0,

effectively equalizing self-influence, while the off-diagonal similarity weights are converted to negative fractional weights that decay based on the total degree of the incident nodes.²²

SciPy's `csgraph.laplacian` handles this complex normalization safely via the `normed=True` parameter, seamlessly supporting both dense NumPy arrays and sparse matrix representations.²³

Name: `graph_laplacian`

Description: Compute the unnormalized or symmetric normalized graph Laplacian.

Source:

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.csgraph.laplacian.html>

License: BSD-3-Clause

Concept type: analysis

Signature: (`adjacency: NDArray, normalized: bool = True`) -> `NDArray`

Pure function boundary: Dense or sparse adjacency array and boolean flag in, Laplacian array out.

Contracts:

- require: `adjacency` is a symmetric 2D array of shape (N, N)
- ensure: if `normalized=True`, the diagonal elements corresponding to non-isolated nodes equal 1.0

Witness: A 2x2 matrix of ones; verify normalized Laplacian diagonal is and off-diagonal is $[-1, -1]$.

Dependencies: `scipy`

CDG stages covered: `spectral_features`, `gnn_preprocessing`, `graph_laplacian`

Node Feature Engineering and Centrality Metrics

Before a spatial Graph Neural Network initiates its message-passing iterations, augmenting the raw input node features with structural topological priors significantly accelerates model convergence and enhances expressive capacity.

Structural Node Degrees

The most fundamental structural prior is the degree of the node. In a topological framework driven by a $(2, E)$ edge list, computing the degree does not require the computationally

expensive step of instantiating the dense (N, N) adjacency matrix. Instead, leveraging the low-level vectorization capabilities of `numpy.bincount` directly on the source or destination

vectors of the edge list yields an $O(E)$ operation. This methodology scales perfectly with graph sparsity.²⁴

If the edge list is defined strictly such that `edge_index` represents source nodes and `edge_index` represents target nodes, executing `np.bincount(edge_index, minlength=num_nodes)` instantly calculates and provides the out-degree vector. Similarly, applying the `bincount` operation to `edge_index` provides the precise in-degree vector.²⁵

Name: `node_degrees`

Description: Compute the in-degree or out-degree of nodes directly from an edge list.

Source: <https://numpy.org/doc/stable/reference/generated/numpy.bincount.html>

License: BSD-3-Clause

Concept type: analysis

Signature: (`edge_index`: NDArray, `num_nodes`: int, `mode`: str = 'out') -> NDArray

Pure function boundary: Edge list, node count, and directionality in; 1D degree array out.

Contracts:

- require: `edge_index` is of shape (2, E)
 - require: `max(edge_index) < num_nodes`
 - ensure: returned array is of shape (`num_nodes`,)
- Witness: Edge list `[, ]` with `num_nodes` 3; verify out-degree is .

Dependencies: `numpy`

CDG stages covered: `node_feature_engineering`, `gnn_preprocessing`

PageRank Centrality via Power Iteration

PageRank algorithms evaluate the transitive topological importance of nodes within a network, mathematically modeling a stochastic random walk across the graph structure. Centrality metrics are highly effective for modeling influence in team interaction graphs (such as March Mania sports analytics) or vast citation networks (OGB MAG240M).²⁷

A pure functional NumPy implementation of this algorithm is optimally achieved via the Power Iteration method.²⁸ To prevent the stochastic random walker from becoming permanently trapped in "sink nodes" (nodes that possess no outbound edges) and to ensure that the

Markov transition matrix remains irreducible, a damping factor β (typically set to 0.85) and a uniform teleportation probability defined as $(1 - \beta)/N$ are mathematically introduced.²⁹

The iterative mathematical update is defined as:

$$r_{t+1} = \frac{1 - \beta}{N} + \beta M r_t$$

where M represents the column-normalized adjacency matrix. The power iteration executes

recursively until the L_1 norm of the difference between successive rank vectors falls below a strictly defined numerical tolerance (e.g., 10^{-6}).³⁰ The functional atom must carefully normalize the input adjacency matrix by column sums prior to initiating the iteration, explicitly patching any columns that sum to zero (identifying the sink nodes) by substituting a uniform probability distribution vector to maintain matrix stochasticity.³⁰

Name: pagerank

Description: Compute the PageRank centrality of all nodes using the power iteration method.

Source: <https://mccormickml.com/2015/09/17/notes-on-pagerank/>

License: Public Domain / MIT equivalent for algorithm

Concept type: analysis

Signature: (adjacency: NDArray, damping: float = 0.85, max_iter: int = 100, tol: float = 1e-6) -> NDArray

Pure function boundary: Adjacency array and scalar parameters in, 1D PageRank array out.

Contracts:

- require: adjacency is of shape (N, N)
- require: $0.0 < \text{damping} < 1.0$
- ensure: sum of output array elements equals 1.0 (within float tolerance)
Witness: A star graph with one center node and 3 peripheral nodes; verify center node holds the highest rank score.
Dependencies: numpy
CDG stages covered: node_feature_engineering, recommender/centrality

Graph Segmentation via Connected Components

Identifying independent connected components within a larger graph is a foundational operation for graph segmentation and clustering. This operation is utilized extensively in tasks ranging from massive-scale e-commerce entity resolution (e.g., matching disparate merchant data in Shopee or Foursquare ecosystems) to molecular fragmentation in biochemical modeling.³²

The computational paradigm for component extraction diverges significantly based on the graph's sparsity profile and its dynamic requirements. The `scipy.sparse.csgraph.connected_components` function is recognized as the most computationally efficient CPU-bound implementation for large, static sparse graphs. It utilizes highly optimized C++ routines—typically Depth-First Search (DFS) or Breadth-First Search (BFS) traversals—operating directly on memory-efficient Compressed Sparse Row (CSR) arrays.³³

However, for dynamically evolving graphs, environments requiring iterative edge additions, or

deployment targets where the SciPy library is strictly unavailable (such as pure JAX or PyTorch tensor deployments), the Disjoint Set Union (DSU) algorithm, commonly known as Union-Find, is structurally required.³⁵ A pure NumPy implementation of the Union-Find algorithm relies on two critical mathematical optimizations to maintain speed: *path compression* (which flattens the internal tree structure during node queries) and *union by rank* (which attaches smaller component trees to the roots of larger ones).³⁶ By implementing these dual optimizations, the amortized time complexity per edge operation approaches $O(1)$, specifically bounded mathematically at $O(\alpha(N))$, where α is the incredibly slow-growing inverse Ackermann function.³⁷

While SciPy remains the indisputable standard for static, one-shot atom execution, an optimized Union-Find implementation holds immense value for streaming graph architectures.

Name: `connected_components`

Description: Extract the connected components of a graph using an optimized disjoint-set or CSR traversal.

Source:

https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.csgraph.connected_components.html

License: BSD-3-Clause

Concept type: analysis

Signature: (`adj_matrix`: NDArray, `directed`: bool = False) -> tuple

Pure function boundary: Adjacency matrix in; integer count of components and 1D label array out.

Contracts:

- require: `adj_matrix` is of shape (N, N)
 - ensure: returned labels array is of shape (N,)
 - ensure: number of unique labels equals the returned component count
- Witness: A graph of 4 nodes with edges (0,1) and (2,3); verify component count is 2 and labels group identically.
- Dependencies: `scipy`
- CDG stages covered: `entity_resolution`, `graph_segmentation`

Complex Graph Extraction: Road Networks and Image Skeletons

The translation of continuous 2D or 3D binary skeleton images—such as overhead satellite imagery of road networks evaluated in the SpaceNet3 datasets, or complex vascular imaging in

biomedical domains—into formalized mathematical topological graphs is a highly intricate CDG stage. The algorithmic process acts as a literal bridge between computer vision matrices and graph topologies, translating unstructured pixel arrays into strict mathematical nodes and spatial edges.³⁸

The standard computational approach relies heavily on precise pixel connectivity classification. Once an input image is fully skeletonized (reduced to 1-pixel wide representations using morphological algorithms like Zhang's iterative thinning method or Lee's octree method⁴⁰), every active pixel in the resultant matrix is systematically evaluated based on its local 8-connected pixel neighborhood (in a 2D context).⁴¹

This classification yields three distinct topological states:

1. **Endpoints (Graph Tips):** Active pixels possessing exactly 1 neighboring active pixel.⁴³
2. **Paths (Graph Edges):** Active pixels possessing exactly 2 neighboring active pixels, forming the continuous segments.
3. **Junctions (Graph Nodes):** Active pixels possessing 3 or more neighboring active pixels, denoting an intersection.³⁹

Advanced Python libraries such as sknw (Skeleton Network) and skan implement this extraction logic with high efficiency.³⁸ The extraction algorithm initiates a comprehensive scan of the binary image to identify all junctions and endpoints, formally initializing them as spatial graph nodes. It subsequently traces the intervening path pixels connecting these nodes to extract the edge morphology.

To be fully functional in a GNN context, the output of this atom must provide not only the topological edge_index matrix but also extensive spatial metadata: the exact Cartesian coordinates of the nodes, the ordered coordinates of the intermediate path pixels, and the physical Euclidean length (the weight) of the constructed edges.³⁸ This multi-modal output bridges convolutional computer vision techniques and graph neural networks, empowering Spatial Graph Convolutional Networks to operate on inherently image-based data structures without losing the geometric nuances of the original visual source.

Name: skeleton_to_graph

Description: Convert a binary skeleton image into a topological graph of junctions and path segments.

Source: <https://github.com/Image-Py/sknw>

License: BSD-3-Clause

Concept type: graph_traversal

Signature: (skeleton: NDArray) -> tuple

Pure function boundary: 2D/3D binary skeleton array in; edge index, edge weights, and spatial

metadata dictionary out.

Contracts:

- require: skeleton is a binary array containing only 0s and 1s
- ensure: output edge_index references valid node indices extracted from the image
Witness: A skeleton of a "Y" shape; verify output graph contains 1 junction node, 3 endpoint nodes, and 3 edges.
Dependencies: numpy, scipy.ndimage
CDG stages covered: computer_vision_to_graph, spatial_network_extraction

Synthesis of the Functional Graph Ecosystem

The comprehensive analysis of architectural formats, algorithms, and memory constraints dictates that the optimal implementation of graph utility atoms for environments like sciona-atoms requires an unyielding adherence to the functional programming paradigm and a profound understanding of hardware acceleration limits. By leveraging low-level NumPy vectorization—such as the memory-efficient execution of `np.bincount`, `np.add.at`, and `cdist`—alongside highly optimized SciPy C++ wrappers like `cKDTree` and `csgraph.laplacian`, the library can consistently achieve mathematically optimal bounds on both time and space complexity.

Crucially, this is accomplished without introducing the severe deployment friction of heavyweight, stateful frameworks such as PyTorch or the object-oriented overhead of NetworkX. The inherent duality of the structural graph format—the message-passing Edge List versus the algebraic Adjacency Matrix—must be accommodated natively through hyper-fast bridging atoms. Ultimately, these meticulously designed functional implementations provide a robust, deterministic, and highly scalable foundation upon which advanced GNN preprocessing and topological feature engineering pipelines can reliably execute at an industrial scale.

Works cited

1. Laplacian matrix - Wikipedia, accessed April 29, 2026, https://en.wikipedia.org/wiki/Laplacian_matrix
2. MDAnalysis: A Toolkit for the Analysis of Molecular Dynamics Simulations - PMC - NIH, accessed April 29, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC3144279/>
3. Nearest Neighbors - Scientific Computing with Python, accessed April 29, 2026, https://caam37830.github.io/book/08_geometry/nearestneighbor.html
4. query_ball_point — SciPy v1.17.0 Manual, accessed April 29, 2026, https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.query_ball_point.html
5. 1.6. Nearest Neighbors — scikit-learn 1.8.0 documentation, accessed April 29, 2026, <https://scikit-learn.org/stable/modules/neighbors.html>
6. scikit-learn/sklearn/neighbors/_graph.py at main · scikit-learn/scikit ..., accessed April 29, 2026,

- https://github.com/scikit-learn/scikit-learn/blob/master/sklearn/neighbors/_graph.py
7. cdist — SciPy v1.17.0 Manual, accessed April 29, 2026, <https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.distance.cdist.html>
 8. Efficient way of vectorizing distance calculation - Stack Overflow, accessed April 29, 2026, <https://stackoverflow.com/questions/55854611/efficient-way-of-vectorizing-distance-calculation>
 9. query_ball_point — SciPy v1.17.0 Manual, accessed April 29, 2026, https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.KDTree.query_ball_point.html
 10. Directional Message Passing on Molecular Graphs via Synthetic Coordinates, accessed April 29, 2026, https://papers.neurips.cc/paper_files/paper/2021/file/82489c9737cc245530c7a6ebef3753ec-Paper.pdf
 11. Geometry-complete perceptron networks for 3D molecular graphs - PMC - NIH, accessed April 29, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10904142/>
 12. Practical Graph Neural Networks for Molecular Machine Learning - Towards Data Science, accessed April 29, 2026, <https://towardsdatascience.com/practical-graph-neural-networks-for-molecular-machine-learning-5e6dee7dc003/>
 13. Getting Started with the RDKit in Python, accessed April 29, 2026, <https://www.rdkit.org/docs/GettingStartedInPython.html>
 14. Computing Distance Matrices with NumPy | Jay Mody, accessed April 29, 2026, <https://jaymody.com/blog/distance-matrices-with-numpy/>
 15. Python Scripting for Computational Molecular Science - MolSSI Education, accessed April 29, 2026, http://education.molssi.org/python_scripting_cms/aio/index.html
 16. Optimising pairwise Euclidean distance calculations using Python - Towards Data Science, accessed April 29, 2026, <https://towardsdatascience.com/optimising-pairwise-euclidean-distance-calculations-using-python-fc020112c984/>
 17. python - Convert adjacency matrix to edge list - Stack Overflow, accessed April 29, 2026, <https://stackoverflow.com/questions/71558233/convert-adjacency-matrix-to-edge-list>
 18. fast conversion from edge list to adjacency matrix in python - Stack Overflow, accessed April 29, 2026, <https://stackoverflow.com/questions/77519582/fast-conversion-from-edge-list-to-adjacency-matrix-in-python>
 19. Tutorial: Normalized Graph Laplacian | by Sik-Ho Tsang - Medium, accessed April 29, 2026, <https://sh-tsang.medium.com/tutorial-normalized-graph-laplacian-f74593feace7>
 20. normalized_laplacian_matrix — NetworkX 3.6.1 documentation, accessed April 29,

- 2026,
https://networkx.org/documentation/stable/reference/generated/networkx.linalg.laplacianmatrix.normalized_laplacian_matrix.html
21. python - Problem in calculating the symmetric normalised laplacian matrix - Stack Overflow, accessed April 29, 2026,
<https://stackoverflow.com/questions/60149225/problem-in-calculating-the-symmetric-normalised-laplacian-matrix>
 22. laplacian — SciPy v1.17.0 Manual, accessed April 29, 2026,
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.csgraph.laplacian.html>
 23. How to return specific nodes with the same degree value in a graph in PyTorch?, accessed April 29, 2026,
<https://stackoverflow.com/questions/68185259/how-to-return-specific-nodes-with-the-same-degree-value-in-a-graph-in-pytorch>
 24. numpy.bincount — NumPy v2.2 Manual, accessed April 29, 2026,
<https://numpy.org/doc/2.2/reference/generated/numpy.bincount.html>
 25. Computing the size of the in-degree histogram tensor dynamically in the PNA example · Issue #3925 · pyg-team/pytorch_geometric - GitHub, accessed April 29, 2026, https://github.com/pyg-team/pytorch_geometric/issues/3925
 26. page rank algorithm - Kaggle, accessed April 29, 2026,
<https://www.kaggle.com/code/vinitkp/page-rank-algorithm>
 27. Understanding and implementing the PageRank algorithm in Python | by Tadashi - Medium, accessed April 29, 2026,
<https://medium.com/@TadashiHomer/understanding-and-implementing-the-pagerank-algorithm-in-python-2ce8683f17a3>
 28. Implementing PageRank using the Power Method - Computational Science Stack Exchange, accessed April 29, 2026,
<https://scicomp.stackexchange.com/questions/3631/implementing-pagerank-using-the-power-method>
 29. Notes on PageRank - Chris McCormick, accessed April 29, 2026,
<https://mccormickml.com/2015/09/17/notes-on-pagerank/>
 30. python implementation of pagerank - GitHub Gist, accessed April 29, 2026,
<https://gist.github.com/1338222/84d767a68da711a154778fb1d00e772d65322187>
 31. union_find - Memgraph, accessed April 29, 2026,
https://memgraph.com/docs/advanced-algorithms/available-algorithms/union_find
 32. cupyx.scipy.sparse.csgraph.connected_components — CuPy 15.0.0a1 documentation, accessed April 29, 2026,
https://docs.cupy.dev/en/latest/reference/generated/cupyx.scipy.sparse.csgraph.connected_components.html
 33. scipy.sparse.csgraph.connected_components — SciPy v0.12 Reference Guide (DRAFT), accessed April 29, 2026,
https://docs.scipy.org/doc/scipy-0.12.0/reference/generated/scipy.sparse.csgraph.connected_components.html
 34. deehzee/unionfind: A union-find disjoint sets data structure implemented in

- Python with the "Weighted Quick Union with Path Compression" algorithm. - GitHub, accessed April 29, 2026, <https://github.com/deehzee/unionfind>
35. Mastering Dara Algorithm — Part 28 Understanding Union Find in Python | by Connie Zhou, accessed April 29, 2026, <https://medium.com/@conniezhou678/mastering-dara-algorithm-part-28-understanding-union-find-in-python-155da9e04ccb>
 36. Union Find Data Structure - Number of Connected Components in an Undirected Graph (LeetCode) - YouTube, accessed April 29, 2026, <https://www.youtube.com/watch?v=ymxPZk7TesQ>
 37. Image-Py/sknw: build net work from skeleton image (2D-3D) - GitHub, accessed April 29, 2026, <https://github.com/Image-Py/sknw>
 38. ykaranouskaya/Skeleton_to_graph-labeling-: The second way to create the graph from the 3D skeleton. - GitHub, accessed April 29, 2026, https://github.com/ykaranouskaya/Skeleton_to_graph-labeling-
 39. Skeletonize — skimage 0.25.2 documentation, accessed April 29, 2026, https://scikit-image.org/docs/0.25.x/auto_examples/edges/plot_skeleton.html
 40. Construct a graph from the skeleton image of a binary form - DIP4FISH, accessed April 29, 2026, <https://dip4fish.blogspot.com/2014/05/construct-graph-from-skeleton-image-of.html>
 41. Extract graph from skeleton image using OpenCV - GitHub Gist, accessed April 29, 2026, <https://gist.github.com/Nico-Curti/a586e6f58d4a2c758b77a3f4492e6d3f>
 42. Calculating a graph from a skeletonized image - Stack Overflow, accessed April 29, 2026, <https://stackoverflow.com/questions/57029293/calculating-a-graph-from-a-skeletonized-image>
 43. Source code for skan.csr, accessed April 29, 2026, https://skeleton-analysis.org/stable/_modules/skan/csr.html
 44. graph - Handle package sknw in python - Stack Overflow, accessed April 29, 2026, <https://stackoverflow.com/questions/46601739/handle-package-sknw-in-python>